Blink 和 Rust 混合编程

沿 BLEngine 的真实代码从算法走进复杂项目

B 表达计算关系

Rust 组织资源与寿命

标准阅读版

2026 年 10 月 25 日

Blink v0.1

目录

1 两种语言，一条完整的数据流 5

2 第一次运行：让 Rust 驱动真实的 B 算法 7

3 数据布局：两边说的是同一件东西 12

4 驻留与时间：连续演化只交接一次 15

5 分支与汇合：让依赖成为程序的形状 18

6 共享资产：view 连接数据，也连接寿命 21

7 空间层级：Rust 管树，B 算变换 24

8 从一个结果到多个消费者 27

目录 · 续

9 材质计算：让数学留在 B，让资产留在 Rust 30

10 效率：算得快，也要交接得少 34

11 资产读取：让 Rust 并发服务于真实消费者 37

12 一帧渲染：把资源交出去，再完整接回来 40

13 完整项目：生成一幅材质图 44

14 走向复杂项目：保持边界，增加能力 50

附录 A 完整命令与运行记录 52

附录 B 源码阅读路线与摘录范围 54

写在前面

面向读者：读过《并行编程的艺术》，会写 C/C++ 或基础 Rust，希望将 Blink 用于实际项目的人。

版本：2026 年 10 月 25 日 • Blink v0.1。

第一本书讨论怎样表达并行计算。这一本继续追问：算法算完以后，数据交给谁？下一阶段什么时候可以开始？模型同时被多个对象使用时，谁负责保存它？程序退出时，尚未完成的工作怎么办？

这些问题决定了一个漂亮的算法能否成为一个可靠的项目。Blink 和 Rust 的结合，正好让计算关系与资源关系相互配合：B 把可并行的工作写清楚，Rust 把数据的归属、交接和寿命写清楚。

Blink 简称 B，也寓意 Base。它希望成为 AI 超算时代的基础语言。这一愿景要落在日常工程里：一个已有的 Rust 项目，应当能够逐步采用 B 算法，保留自己的业务模型，并把计算带到 CPU 多核或 GPU。

Blink 开源项目、两本书及配套源码：

https://github.com/sgua83/Blink

一本围绕真实调用链展开的书

本书以 BLEngine 的实现为主线，从粒子模拟出发，逐步进入场景层级、组件、材质、资产读取和渲染输出。每次只引入理解当前问题所需的概念，图形学公式也只解释到能够读懂数据流。

引擎摘录固定为截至 P4 CL12965 的已提交源码；这是一份可追溯的学习样本。段落中的代码节选会重排换行、略去无关字段和重复打包过程，必要的错误传播与资源寿命仍予保留。配套 Programs 中的引擎文件保留原文，来源清单列出版本和 SHA256；book 模块是明确标注的教学组合。

本书的独立 Rust 宿主位于 sample/hybrid。它只依赖 Blink 的两个 crate，读者下载 Blink 后即可运行，无需安装 BLEngine 或配置 P4。深入渲染和资产管理的章节阅读真实引擎节选；最终动手项目则把真实材质与显示转换串起来，输出一幅一百万像素的材质图。

阅读路线

阶段	章节	读完能做什么
建立配合	1-3	编译 B、打包数据、在 Rust 中执行
组织依赖	4-6	复用驻留数据、衔接提交、共享只读资产
进入工程	7-9	处理层级、组件与纹理材质
管好成本	10-12	解释计时、组织 I/O、归还输出资源
独立完成	13-14	生成图像，设计自己的混合项目

1 两种语言，一条完整的数据流

1.1 先画出责任，再写接口

一个粒子模拟项目里，有两类性质不同的工作。

第一类是计算：根据位置求加速度，更新速度和位置，再统计能量。粒子之间大部分工作互不依赖，统计则需要汇合。

第二类是组织：决定读取哪个文件、创建多少粒子、模拟多长时间、选择哪台设备，以及什么时候保存结果。这些工作需要清楚的控制流和资源归属。

BLEngine 将前一类放在 B，后一类交给 Rust。这个分工沿着实际职责展开。循环是不是很短，或者某个函数是否容易翻译成另一种语言，都不足以决定它应该放在哪里。



图 1-1 B 决定算什么及阶段间的依赖；Rust 持有实际存储，并把它交给执行框架。

1.2 从引擎里取出最短的主线

BLEngine 的 simulation/statistics.b 将两个计算连接起来。保留原有名称，只调整换行：

B / 语言代码

```
program step_and_total(
    inout dataset<BLBody<float3,3>>
        bodies,
    write dataset<BLTotal,1> total,
    f32 dt, f32 strength
) on compute {
    step(bodies,dt,strength);
    total_energy(bodies,total);
}
```

读成一句话：先推进所有粒子，再对更新后的能量求和。dt 是一个物理时间步；strength 是中心力场的强度。total 只有一条记录，保存所有粒子的总能量。

Rust 对应的主干也很短：

Rust / 宿主代码

```
let program = CpuProgram::new(program)?;
program.run(&mut data, &values)?;
```

第一行准备原生 CPU 程序；第二行借用实际数据执行。算法没有复制到 Rust 中，Rust 也不需要替 B 的每一个粒子创建线程。

1.3 理解两个方向的约束

B 中的 read、write、inout 表达计算访问。read 允许读取，write 要求产生完整输出，inout 允许读取旧值再写回。Rust 的 &、&mut 和所有权移动约束宿主怎样使用对象。它们服务于同一条数据流，但分工不同。

比如一个 Rust 变量可以仍然存在，而它代表的 GPU 工作尚未完成；拥有变量不等于可以立即读取设备结果。执行框架需要把真实完成点和存储一起保存。

反过来，B 能表达一个只读数据集，也不意味着宿主可以把任意裸地址包装成这个数据集。实际存储、布局与引用来源仍要由 Rust 和执行框架提供。

1.4 混合编程的美，在交接处

一个清楚的项目能够回答四件事：数据由谁创建，现在由谁持有，下一个消费者是谁，何时不再需要它。

后面的章节始终沿着这四个问题前进。看到一个 Vec、一个 view、一个 Submission 或一条通道时，先找它连接的真实生产者与消费者，再研究具体 API。

2 第一次运行：让 Rust 驱动真实的 B 算法

2.1 下载、编译和运行

动手环境为 Windows x64。安装 Git for Windows、Rust stable 的 MSVC 工具链，以及 Visual Studio 的“使用 C++ 的桌面开发”组件。打开 x64 Developer PowerShell，执行：

PowerShell / 命令行

```
git clone `
    https://github.com/sgua83/Blink.git
Set-Location Blink
cargo build --workspace --release

$env:CARGO_TARGET_DIR = `
    Join-Path $PWD 'target'
$manifest = 'sample/hybrid/Cargo.toml'
cargo build --release `
    --manifest-path $manifest

$book = `
    'target/release/blink-rust-book.exe'
& $book particles cpu 3
```

本书后续命令都在 Blink 仓库根目录执行，并沿用 $book。Cargo 构建 Rust 宿主；宿主运行时由 blanguage 编译所选 B 入口，再由 blcompute 执行。没有第三方 Shader 编译器或并行运行时。

CPU 命令不要求显卡；GPU 命令要求驱动提供相应 Vulkan 能力，无需额外安装 Vulkan SDK。当前构建与运行范围是 Windows x64，不能把这些命令视为其他平台已完成适配的证明。



图 2-1 终端输出排印，数值来自本书配套宿主的真实运行。计时受机器和运行状态影响。

三个粒子的初始 x 分别是 1、1.001、1.002，初始速度都是 (0,1,0)。一步为 0.01 秒。中心力使 x 略微减小，向上的速度使 y 增至 0.01；总能量接近 1.5。

这里用三个粒子，是为了逐项理解结果。后面的一百万采样点材质案例承担性能对照；小数据的正确性示例不承担 GPU 吞吐量演示。

2.2 先认清三个 Rust 写法

熟悉 C/C++ 的读者，只需先抓住以下写法：

Rust / 宿主代码

```
let mut data = vec![bodies, total];
program.run(&mut data, &values)?;
println!("完成");
```

let 绑定一个名字，mut 允许修改；vec! 构造拥有元素的动态数组。bodies 和 total 被放入 Vec 后，所有权随之转移，旧变量不能继续独立使用它们。&mut data 是一次独占的可修改借用。

末尾的 ? 表示：成功就取出结果，失败就把错误交给调用者。因此“完成”只在运行成功后打印。它没有把失败替换成零或空数据。

本书的 main 返回 blcompute::Result<()>。I/O、编译、设备和执行错误沿同一条 Result 路径返回；实际业务可在顶层增加合适的上下文。

2.3 从文件得到一个可执行入口

下面摘自配套 main.rs 的加载函数：

Rust / 宿主代码

```
let root = Path::new(
    "sample/hybrid/Programs"
);
let entry = root.join(
    "simulation/statistics.b"
);
let mut compilation =
    blanguage::compile_entry(
    &entry,
    root,
    "step_and_total",
)?;
let program =
    compilation.programs.remove(0);
```

最后一行取出这个已明确选定、无前置条件的具体程序。ExecutableProgram 代表编译器完成检查之后得到的结果，普通应用不应从任意外部字节伪造它。

模块根告诉编译器怎样找到 simulation.bodies、math.field 等导入。调用者选择的是公开 program 的名字，函数体的组合仍由 B 模块维护。

2.4 创建存储，填入输入，交给 CPU

完整准备过程分为四步：

Rust / 宿主代码

```
let bodies =
    Dataset::new(&program, 0, 3)?;
let total =
    Dataset::new(&program, 1, 1)?;
let mut data = vec![bodies, total];
let values = [
    Value::F32(0.01),
    Value::F32(0.2),
];
```

第 0 个数据绑定是 bodies，第 1 个是 total。uniform 按参数顺序提供 dt 和 strength。实际初值填充见下一章；Dataset::new 按程序给出的记录布局创建存储。

Rust / 宿主代码

```
let program = CpuProgram::new(program)?;
program.run(&mut data, &values)?;
```

run 返回时 CPU 调用已经结束，Rust 可以读取 data。这个同步入口适合作为第一步。先把一次调用讲清楚，再学习 GPU 的提交与完成对象，能够避免同时面对太多新概念。

2.5 B 内部究竟执行了什么

simulation/bodies.b 的 step 先调用 integrate，再调用 energy。integrate 的核心是：

B / 语言代码

```
parallel (body in bodies) {
    integrate_body(body,dt,strength);
}
```

其中每个粒子的计算为：

B / 语言代码

```
body.velocity +=
    central_acceleration<Power>(
        body.position,strength
    )*dt;
body.position += body.velocity*dt;
```

随后 energy 对每个粒子计算 0.5*dot(velocity,velocity)，total_energy 再执行 reduce_sum。前两个阶段各自允许粒子间独立推进；归约阶段以汇合的方式产生一个总量。



图 2-2 每个阶段内部展开元素工作；阶段之间保持数据依赖。示意图不表示固定线程数量或耗时。

Rust 的一次 run 并不等于 B 的一个 parallel。它可以包含多个计算阶段。把这些阶段写成 program，编译器才能看见完整的先后关系。

3 数据布局：两边说的是同一件东西

3.1 相同字段名，不代表相同内存

B 的 BLBody<float3,3> 包含 position、velocity、energy。不要直接假定 Rust 的三个数组字段与它具有相同的字节布局，即使写上 repr(C) 也不足以证明与 B 的目标布局一致。

本书运行中得到：position 偏移 0 字节，velocity 偏移 16 字节，energy 偏移 28 字节，记录步长 32 字节。position 的三个 f32 占 12 字节，之后有四字节间隔。

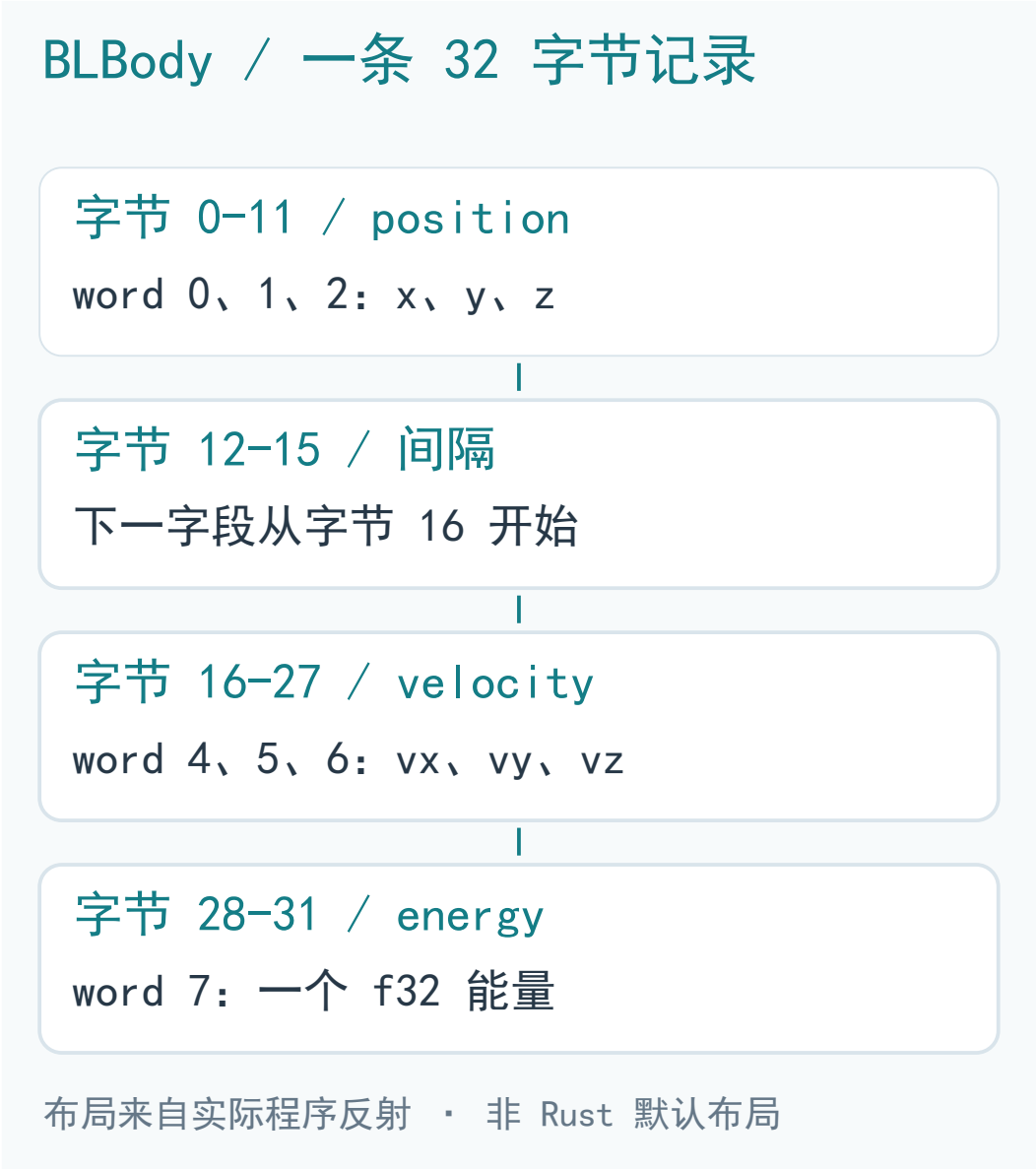


图 3-1 word 是一个 u32 存储单元，四字节。字段偏移由程序布局查询得到。

3.2 查询布局，再填数据

BLEngine 的 Rust 模拟宿主按字段偏移和步长打包。配套 particles.rs 延续同样的写法：

Rust / 宿主代码

```
let stride =
    bodies.stride() as usize / 4;
let position = bodies
    .field_offset("position")?
    as usize / 4;
let velocity = bodies
    .field_offset("velocity")?
    as usize / 4;

for (i, row) in bodies.words_mut()
    .chunks_exact_mut(stride)
    .enumerate()
{
    let x =
        1.0 + (i % 1000) as f32*0.001;
    row[position] = x.to_bits();
    row[velocity + 1] =
        1.0_f32.to_bits();
}
```

to_bits 保留 f32 的位表示，写入底层 u32。它与 `x as u32` 不同：后者进行数值转换，会把 1.5 变成整数 1。读取时使用 f32::from_bits 恢复浮点值。

stride 和偏移只查一次，实际填充循环直接使用结果。没有必要在每个粒子的每个字段处重新查询布局。

3.3 Dataset 的意义超过一块内存

Dataset 记录元素类型、步长、数量及实际存储。带 view 的数据还保留来源。用于有效前缀的 DomainBindings 则进一步携带生产与消费关系。

因此，把 bytes 和 count 重新拼在一起，不能自动恢复原来的域关系。需要继续使用某个筛选结果时，应保留对应的 DomainBindings；into_datasets 表示主动取出裸数据并放弃这类关系。

本书粒子案例使用完整数据域，所以 Vec<Dataset> 已足够。只有确有有效前缀的算法，才引入相关关系，避免在第一个例子中制造不必要的概念。

3.4 把同一份程序交给 GPU

CPU 与 GPU 路径中的 data、program、values 沿用相同名字：

Rust / 宿主代码

```
let device =
    Device::for_program(&program)?;
let data = data.into_iter()
    .map(|d| device.upload(d))
    .collect::<Result<Vec<_>>>()?;
let program = device.compile(program)?;

let done = program
    .submit(data, &values)?
    .wait()?;
let bodies = device.download(
    &done.datasets()[0]
)?;
```

upload 消费主机 Dataset，产生设备数据。submit 消费设备绑定，返回 Submission；wait 返回 Completed。download 才把需要的设备结果复制到主机。

PowerShell / 命令行

```
& $book particles gpu 3
```

得到的第一粒子位置与 CPU 相同到这里显示的小数位，总能量为 1.500003。设备创建、上传、提交和读回都有实际成本；三个粒子的 GPU 时间不能用来评价大规模算法吞吐量。

3.5 第一次遇见 move

submit 接收 data 后，Rust 不允许继续使用原来的 data。这种“移交”很像把一叠资料交给负责处理的人：自己保留一张写着地址的纸，不会凭空多出第二份可以任意修改的资料。

完成对象把数据交回来，或者把它继续交给下一阶段。这个写法让“谁正在使用数据”直接体现在代码中，也为后面的异步组合打下基础。

4 驻留与时间：连续演化只交接一次

4.1 先区分时间步和测量次数

粒子模拟每推进一次，位置都会改变。三个 dt=0.01 的时间步代表物理世界演化了 0.03 秒。这里的三次有算法意义，与为了计时反复运行同一输入不同。

如果每一步都下载全部粒子，再重新上传下一步，数据会在 CPU 与 GPU 之间来回旅行。算法需要上一时刻的位置，未必需要 CPU 在每一步都看到它。

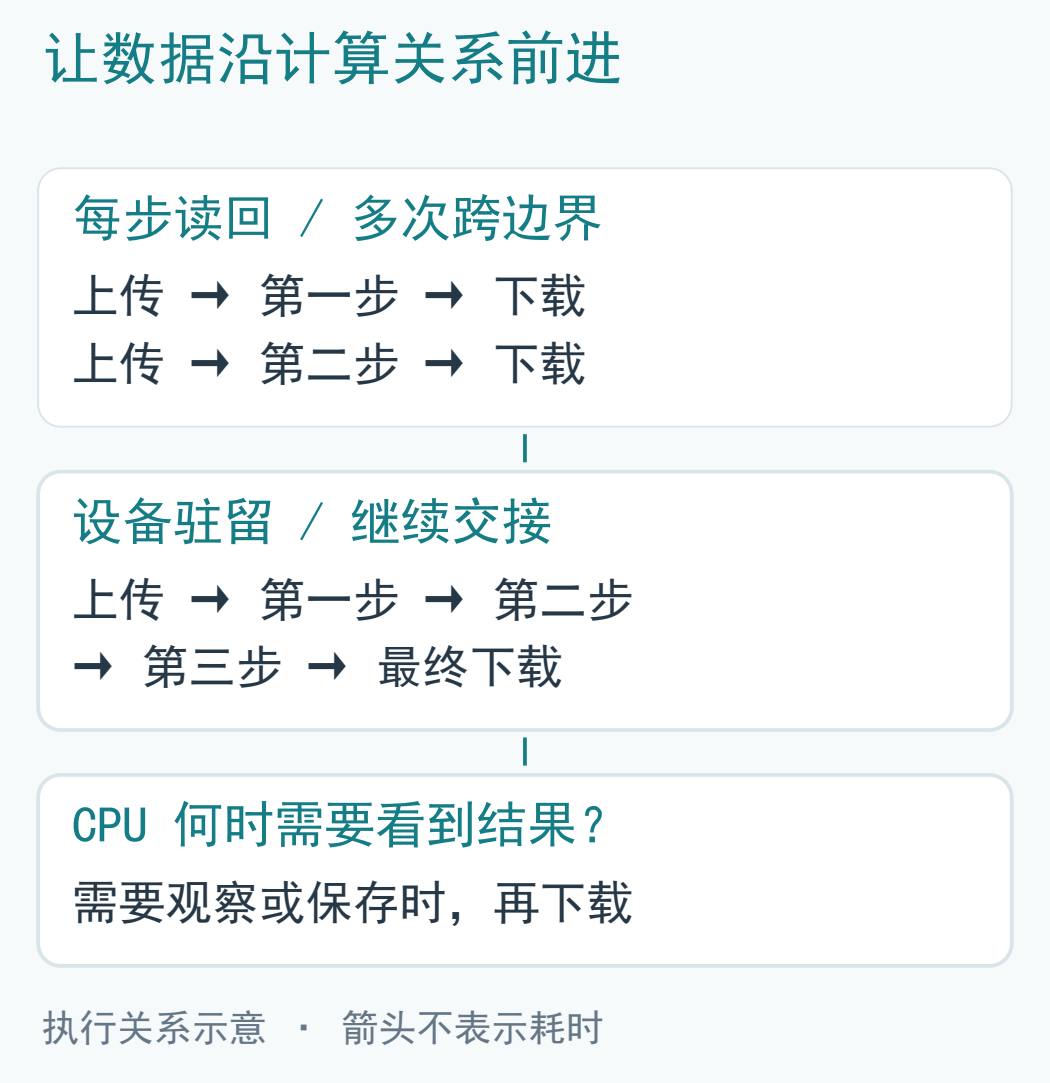


图 4-1 上方每步经过主机；下方在设备上衔接，只在需要观察时读回。箭头描述传输与依赖，不代表实测长度。

4.2 沿用同一组绑定

配套 steps 命令使用三个独立提交，展示所有权的衔接：

Rust / 宿主代码

```
let first =
    program.submit(data, &values)?;
let second = program.submit_after(
    first, &[0, 1], vec![], &values
)?;
let third = program.submit_after(
    second, &[0, 1], vec![], &values
)?;
let done = third.wait()?;
```

order 的 [0,1] 表示下一步继续接收上一项的 bodies 与 total；vec![] 表示没有新增数据。前两步没有调用 wait，数据仍由执行框架持有，后一步依赖前一步的真实完成关系。

同队列提交的衔接不意味着后一步可以在前一步写完之前读取同一粒子。允许主机先行组织工作，与允许违背计算依赖，是两件事。

4.3 一次提交包含多个物理时间步

BLEngine 的 Rust 模拟入口还使用 submit_steps：

Rust / 宿主代码

```
let steps = NonZeroU32::new(3).unwrap();
let done = program
    .submit_steps(data, &values, steps)?
    .wait()?;
```

当每一步运行同一程序、使用同一组 uniform 时，这个入口可以把多个物理时间步放进一次设备提交。若每一步需要不同的 dt 或其他程序，就应表达实际变化，不能把它们硬塞进同一个不变参数调用。

submit_after 适合学习阶段组合，submit_steps 适合同一演化程序的连续推进。选择依据是算法的真实结构。

4.4 编译、执行和解释结果

PowerShell / 命令行

```
& $book steps
```



图 4-2 第一粒子的 y 接近 0.03，x 在中心力作用下继续减小。

初始速度沿 y 轴；演化后 y 已不是三个完全相等的增量，因为力场也在改变速度。最终总能量为 1.499899。这个量是此离散更新算法计算的动能总和，不是宣称其为严格守恒的完整机械能。

4.5 资源什么时候可以释放

主机结束当前循环，并不意味着设备完成了当前工作。执行框架把实际存储与最后使用点关联起来，让提交需要的数据继续存活。只有不再有消费者，且相关设备使用已经结束，物理资源才适合释放。

应用通常不需要再建立一份“可能正在使用的 buffer 列表”。沿着现有 Submission、绑定和完成对象传递所有权，就能保持一条清楚的生命周期。

5 分支与汇合：让依赖成为程序的形状

5.1 两个群体，一个统计结果

考虑两组独立粒子。left 与 right 各自更新并统计总能量，最后把两个总量相加。左组初速度为 (0,1,0)，右组为 (0,2,0)。动能与速度平方成正比，右组的总能量应约为左组的四倍。

两个群体的计算互不读取对方的数据；汇合阶段则必须等待两个结果都可用。



图 5-1 分支表达逻辑独立性。当前 API 使用同一设备队列，不承诺两个分支在不同硬件队列同时执行。

5.2 最小的 B 汇合程序

book/handoff 是本书唯一负责这个演示的教学模块。它复用引擎的 BLTotal 类型：

B / 语言代码

```
program finish(
    read dataset<BLTotal,1> left,
    read dataset<BLTotal,1> right,
    write dataset<BLTotal,1> combined
) on compute {
    parallel (value in combined) {
        value.value =
        left[0].value+right[0].value;
    }
}
```

三个数据集的静态长度都是 1，下标 0 位于已知范围内。宿主按接口创建相应的一条记录，B 直接读取这两个值。层级章节还会讨论动态父索引与场景结构的关系。

5.3 Rust 把真实结果交到一起

Rust / 宿主代码

```
let left =
    program.submit(left, &values)?;
let right =
    program.submit(right, &values)?;
let done = finish.submit_join(
    [left, right],
    &[1, 3, 4],
    vec![combined],
    &[],
)?.wait()?;
```

先把两项提交的数据按参数顺序连接，再追加 combined，索引含义如下：

索引	实际数据	是否交给 finish
0	left.bodies	否
1	left.total	是，第 0 绑定
2	right.bodies	否
3	right.total	是，第 1 绑定
4	combined	是，第 2 绑定

order 只选择并排列真实数据，不复制粒子数组。未继续使用的粒子存储仍遵守实际设备使用的完成关系，不会因为没有出现在新参数中就被过早释放。

5.4 运行与读数

PowerShell / 命令行

```
& $book flow
```



图 5-2 left 与 right 来自各自的 step_and_total，combined 由 B 汇合程序产生。

不能把两个 producer 的逻辑独立性直接换算成“两倍速度”。真正的收益来自避免不必要的主机等待、读回和重新上传；实际硬件重叠取决于队列、调度和工作负载。

5.5 再往前一步：保留域关系

当分支由 compact 产生有效前缀时，应通过 Submission::partition、into_bindings 和 DomainBindings 延续原有关系。它们移交数据及其关系；手动读取一个 count，再重新拼 Vec，不等于继承编译器已建立的生产关系。

先学会完整域上的汇合，再把同一套所有权思想用于有效域，可以把复杂性留在真正需要它的地方。

6 共享资产：view 连接数据，也连接寿命

6.1 一千个实例，不应有一千份模型

场景中的许多对象可以使用同一个模型，只改变变换、材质或颜色。如果每个实例都复制全部三角形，内存成本会随着实例数增长，更新和释放也会变得复杂。

BLEngine 的 BLInstance 保存四种只读 view，而不是把所有几何内容展开到实例记录中。摘自 render/trace.h：

B / 语言代码

```
struct BLInstance {
    view<BLTriangle> triangles;
    view<BLNormal> normals;
    view<BLTextureTriangle> texcoords;
    view<BLMaterialSlot> materials;
    float4 tint;
};
```

每条实例记录表示“使用哪些数据”，实际模型仍由资产存储拥有。两条实例引用同一份 normals，二者就分享这份实际来源。



图 6-1 实例记录很小；模型内容沿只读 view 共享。资源引用连接到真实所有者。

6.2 Rust 先创建来源，再冻结

src/scene.rs 中，宿主用反射的 view 元素类型创建三角形、法线和纹理坐标数据。以下为三角形来源的主干：

Rust / 宿主代码

```
let triangles = Dataset::for_view(
    program, 2, "triangles", count
)?;
// 按字段偏移填入实际三角形数据。
let triangles = device
    .upload(triangles)?
    .freeze();
```

这里的 2 是实例数据集在该 frame 中的绑定编号，"triangles" 指明要使用哪一个 view 字段的元素类型。freeze 把可移动的数据变成可共享的只读来源。

CPU 的 ReadDataset 使用 Arc；GPU 的 ReadDataset 使用 Rc 持有设备对象。clone 是共享所有权，不是复制整个模型。GPU 对象的线程归属也没有因此改变，不能把 Rc 设备对象随意送进 I/O 工作线程。

6.3 把来源接到实例记录上

下面节选自同一个 shading 函数，保留一个 view 的装配过程；另外三个字段使用相同方式：

Rust / 宿主代码

```
views.push(ViewBinding {
    record: ordinal as u32,
    field: "triangles",
    source: asset.triangles.clone(),
});

device.upload_with_views(data, views)
```

record 说明这是哪一条实例，field 说明接到哪个字段，source 提供真实的只读存储。upload_with_views 将根记录与来源一起交给设备执行框架。CPU 端对应的是 Dataset::with_views。

对照传统裸指针接口时，重要差别不只是少写了一个地址。这里同时保留了类型、范围和来源所有权，实际使用者需要的资产不会仅因宿主局部变量离开作用域就消失。

6.4 跨设备传输不是复制地址

CPU 指针不能直接当作 GPU 地址，GPU 地址也不能直接交给 CPU 解引用。对于带 view 的数据，传输需要复制实际闭合引用图，并在目标端建立正确的地址关系。

只把根记录的 bytes 保存到文件，再原样上传到另一台设备，不会恢复它引用的模型。保存资产时应保存正式格式中的内容和关系；重新加载时由实际所有者重建运行时资源。

6.5 能共享什么，要看谁会修改

模型三角形和法线适合不可变共享；每个实例独立变化的 tint、变换或选择状态，应留在实例自身或相应业务所有者处。把所有东西都冻结，会让必要更新变成重新创建；把所有东西都做成可变共享，又会引入不必要的同步。

因此先找“谁写、谁读、是否会变化”，再决定存储形式。view 的简洁来自正确的数据关系。

7 空间层级：Rust 管树，B 算变换

7.1 一个能手算的场景

根节点平移 (10,0,0)，子节点相对根平移 (0,2,0)，孙节点相对子节点平移 (0,0,3)。三个节点都没有旋转和缩放。

于是根的世界位置是 (10,0,0)，子节点是 (10,2,0)，孙节点是 (10,2,3)。这个简单例子足以看清层级的组合方向。



图 7-1 每个输出独立沿父链读取。不同输出可以并行，单条父链内部仍有先后依赖。

7.2 Rust 的场景与 B 的计算快照

BLEngine 的 Rust 场景拥有节点、父子关系、资产引用和节点身份。BLNodeId 包含 index 和 generation：槽位可以复用，但旧身份不能误指新对象。修改父关系时，场景所有者维护无环结构。

计算前，Rust 把当前节点状态打包成连续的 BLHierarchyNode 数据。B 接收的是本次计算所需的快照，不接管编辑命令、历史状态或节点身份的管理。

B / 语言代码

```
struct BLHierarchyNode {
    float3x4 local;
    u32 parent;
    bool has_parent;
    bool active;
};
struct BLWorldTransform {
    float3x4 matrix;
};
```

parent 是快照中的数组位置，不能直接当作跨编辑操作长期有效的节点身份。快照布局和场景 API 各有自己的职责。

7.3 B 如何求一个世界变换

下面为 scene/hierarchy.b 的 resolve 主体：

B / 语言代码

```
parallel (world in worlds) {
    BLHierarchyNode node =
        unsafe_read(nodes,index());
    float3x4 matrix = node.local;
    if (node.active) {
        while (node.has_parent) {
        node = unsafe_read(
        nodes,node.parent
        );
        matrix = compose_affine(
        node.local,matrix
        );
        }
    }
    world.matrix = matrix;
}
```

compose_affine(parent,child) 将父变换放在左边。先由自己的局部空间到父空间，再由父空间继续向上，最后得到世界空间。

串行写法是对每个节点依次执行同一段父链遍历；B 将各个输出展开。算法没有依赖“上一个节点已经写好 worlds”，所以输出间不需要按节点编号排队。

代价也很清楚：同一条祖先链可能被多个输出重复读取。深链会增加工作量；不能仅凭 parallel 就宣称比任何层级算法都更快。是否换成另一种算法，取决于真实场景结构和测量。

7.4 unsafe_read 的前提由谁负责

动态父索引的前提是：每个实际父索引都指向快照中的有效记录，且父关系无环。无环保证循环会到达根；有效索引保证读取在实际存储范围内。

这些关系属于场景结构的真实边界，应由产生场景和快照的所有者维护。为每次祖先读取临时增加一个任意“最多遍历若干层”的限制，会截断合法场景，也会掩盖坏结构。

书中省略节点容器的重复访问细节，但不能把“父链有效”这一责任一起删掉。简洁的主干应建立在明确前提上。

7.5 运行并解释三个输出

配套 scene 命令执行下一章的 evaluate，内部先调用 resolve，因此可以同时观察层级与依赖它的组件。

PowerShell / 命令行

```
& $book scene cpu
& $book scene gpu
```



图 7-2 CPU 与 GPU 得到相同的三节点平移、相机位置和点光源位置。

8 从一个结果到多个消费者

8.1 世界变换只求一次

场景里的网格、相机和灯光都依赖节点的世界变换。如果各自维护一份权威变换缓存，修改父节点时就需要同步多个状态。

BLEngine 的 evaluate 让 world transforms 成为一次计算的共享结果。摘自 scene/components.b：

B / 语言代码

```
resolve(nodes,worlds);
cameras(
    worlds,local_cameras,world_cameras
);
lights(
    worlds,local_lights,world_lights
);
```

这是 evaluate 的计算正文，不是三条 Rust 调度命令。依赖关系在同一 B program 里清楚可见：相机和灯光都读取刚算出的 worlds。



图 8-1 一个场景事实源，一次世界变换计算，多个实际消费者。图不承诺当前后端同时发射两个后续阶段。

8.2 点与方向为什么使用不同的 w

相机挂在孙节点上，本地 eye=(0,0,1)，target=(0,0,0)，up=(0,1,0)。components.b 的主干为：

B / 语言代码

```
value.eye =
    world*float4(value.eye,1.0);
value.target =
    world*float4(value.target,1.0);
value.up =
    world*float4(value.up,0.0);
```

位置要受到平移，所以 w=1；方向不应被平移，所以 w=0。运行后 eye=(10,2,4)，target=(10,2,3)，up 仍为 (0,1,0)。

这段代码体现了混合项目里一种很有用的分工：Rust 决定哪个相机挂在哪个节点，B 执行统一的空间变换。宿主不需要再写一遍相机矩阵算法。

8.3 灯光的分支是算法含义

点光源的位置需要变换；方向光只需要变换方向；面光源还需要变换两条边。真实实现用 kind 区分这些计算。

B / 语言代码

```
if (value.kind!=0) {
    value.position =
        world*float4(value.position,1.0);
}
if (value.kind==0 || value.kind==2) {
    value.direction =
        world*float4(value.direction,0.0);
}
```

这种分支决定不同光源的数学语义，属于算法本身。化简代码时应保留它，不能把所有 if 都归为冗余校验。

8.4 Rust 只打包实际参与的组件

src/components.rs 收集当前节点上存在的相机和灯光，分别创建局部输入与世界输出。没有相机的节点不需要伪造一个默认相机，占据无意义的工作量。

配套场景中只有一台相机和一盏点光源。它们的局部记录各有一个 node 字段，指向自己的场景节点；worlds 仍按全部节点的范围创建。

这说明数据集的长度可以不同：三个节点、一台相机、一盏灯，各自表达自己的工作域。不能因为“这些都属于场景”就强行塞进同样大小的数组。

8.5 结果如何回到业务

真实引擎将计算结果解码为 placements、cameras、lights，再交给实际消费者。编辑语义仍由场景拥有；一次求值结果不应成为第二棵可独立修改的场景树。

上一章命令还会输出点光源的位置 (11,2,0)：它挂在子节点，本地位置为 (1,0,0)。相机 up 仍为 (0,1,0)，方向没有加上平移。

9 材质计算：让数学留在 B，让资产留在 Rust

9.1 从几何位置走向表面响应

前面的例子移动物体。这一章给表面一点光，观察它如何反射。

BLShadingPoint 记录 normal、view、light、uv、lod、tint、roughness、ior 和 radiance。normal 表示表面朝向，view 与 light 表示观察和入射方向，uv 用来读取纹理，roughness 控制表面微观粗糙程度，ior 是折射率参数。

Rust 负责加载或创建纹理、准备数据集并选择设备。B 根据这些输入计算每个采样点的 radiance。

9.2 真实 shade 的正文很短

摘自 material/textured.b，变量名保持原文：

B / 语言代码

```
parallel (point in points) {
    point.radiance = textured_ggx(
        albedo, point.uv, point.lod,
        point.normal, point.view,
        point.light, point.tint,
        point.roughness, point.ior
    );
}
```

与之对应的串行组织只是换成依次遍历 points，仍使用同名 point 和同一公式。差别在于如何组织独立采样点，不在于把变量名或数学改得更复杂。

textured_ggx 先进行 texture_sample，再把纹理颜色乘 tint，交给 material/surface.b 的 ggx_surface。纹理对象作为只读资源进入计算，而不是把纹理复制到每条点记录里。



图 9-1 同一份纹理被许多采样点读取；各点只写自己的 radiance。

9.3 不必先成为图形学专家

GGX 描述由许多微小表面组成的高光分布。理解本章所需的直觉是：法线决定表面朝向，观察和光照方向决定哪些微小表面能够反射到眼睛，粗糙度决定反射集中还是分散。

真实函数先计算：

B / 语言代码

```
f32 nl = dot(normal,light);
f32 nv = dot(normal,view);
if (nl<=0.0 || nv<=0.0) {
    return float3(0.0);
}
float3 half_vector =
    normalize(view+light);
```

背向光或背向观察方向时返回零，是模型的可见性条件。half_vector 位于光与观察方向之间，后续公式用它计算微表面的分布与遮蔽。

最终结果包含漫反射与镜面项：

B / 语言代码

```
return base*(
    (1.0-fresnel)*nl/3.14159265359
)
    + float3(specular);
```

不需要在 Rust 中再实现同一个 GGX 函数。让公式只有一个正式来源，CPU 与 GPU 后端就可以消费同一份算法。

9.4 同一个 Rust 资源模型，两条执行路径

CPU 路径向 run_owned 提交主机纹理：

Rust / 宿主代码

```
let output = program.run_owned(
    cpu::Bindings {
        datasets: vec![points],
        textures: vec![albedo],
    },
    &[],
)?;
```

GPU 路径先上传纹理，提交设备纹理：

Rust / 宿主代码

```
let albedo =
    device.upload_texture(&albedo)?;
let done = program.submit(
    vulkan::Bindings {
        datasets: vec![points],
        accelerations: vec![],
        textures: vec![albedo],
    },
    &[],
)?.wait()?;
```

本例不使用加速结构；后面的光线查询 frame 必须传入真实的场景加速结构。

9.5 准备一张材质采样表

教学 prepare 让法线随采样位置变化，让 roughness 从左到右增加；共享纹理由四个颜色构成。GGX 直接使用引擎原文。

B / 语言代码

```
f32 x=(f32(index()%W)+0.5)/f32(W);
f32 y=(f32(index()/W)+0.5)/f32(H);
point.normal=normalize(float3(
    (x-0.5)*1.4,(y-0.5)*1.4,1.0
));
point.roughness=0.08+0.84*x;
```

W、H 是编译期形状：计时入口处理一千万点，最终图像入口处理一百万像素。

prepare 完整初始化 write 记录，包括暂为零的 radiance；随后 shade 产生新的 radiance。这让每个程序的写入责任清楚，不依赖调用者碰巧把内存清过零。

9.6 编译和原生运行

PowerShell / 命令行

```
& $book sheet cpu
& $book sheet gpu
```

两个命令对同一张一千万点采样表计算相同的准备与 GGX 程序，分别报告首次执行、资源复用后的执行，以及中心点颜色。下一章读这些真实计时，第十三章把结果变成图像。

10 效率：算得快，也要交接得少

10.1 先把比较的两端对齐

sheet cpu 与 sheet gpu 都执行 materials<10000,1000>，都包含采样点准备和真实 GGX 材质计算，都是一千万点。CPU 路径已经使用 Blink 的原生多核执行；这里比较的是 CPU 多核与 GPU 驻留执行，并非拿串行解释器衬托 GPU。

宿主先报告首次运行，再复用同一程序、纹理和存储执行下一次。复用后的计时从调用执行接口开始，到结果完成为止；没有重新编译，也没有把数据传回 CPU。第一次 GPU 等待还可能包含此前尚未完成的上传。

10.2 一次真实运行的时间

本书观测机器：Ryzen 9 9950X，16 核 32 线程，128 GB 内存，AMD Radeon RX 9070 GRE；Rust 1.95.0，release 构建。



图 10-1 终端真实输出节选并排印。两种后端处理同一程序与规模，启动、执行和下载分别计时。

项目	CPU 多核	GPU
准备程序与存储	97.524 ms	629.451 ms，含上传入队
首次执行并等待	45.563 ms	127.771 ms，含未完成上传的影响
驻留后执行并等待	46.684 ms	16.701 ms
GPU 设备时间戳	不适用	13.220 ms
全部点记录下载	不需要	2620.171 ms

在这次观测中，资源复用后的完整执行等待由 46.684 ms 降至 16.701 ms，约快 2.80 倍。这个比较将 GPU 的主机提交和等待也算在内。

13.220 ms 是设备时间戳覆盖的程序区间，不能拿它冒充整个调用耗时。数据集每条 112 字节，一千万条约 1.12 GB；下载整份记录远比一次 GGX 计算昂贵。

10.3 为什么第一次调用不能代表长期成本

只运行一次时，要支付设备创建、原生代码或 pipeline 准备、存储及传输成本。GPU 冷调用在这个例子里明显更贵；连续使用已准备的资源，才有前面测得的计算优势。

这不是把开销藏起来。表中列出的是不同生命周期的真实成本：程序和纹理通常长于一次调用，输入和输出则随实际工作更新。应用应按自己的调用次数分摊一次性成本。

不要每帧重新创建 Device、编译同一 program，或者上传相同纹理；这些对象已经有明确的长期消费者，可以由项目中的实际所有者持有。

10.4 最大的优化可能是少读一些

材质检查命令为了观察计算结果，下载了完整点记录。实际渲染只需要最终像素，不需要 CPU 再拿到每个点的 normal、view、light 和中间数据。

第十三章会让 B 在设备上继续完成显示转换。对于同样一百万像素，完整点记录是 112 MB，最终 BGRA 图像只有 4 MB；下载对象减少为原来的二十八分之一。这个比例来自实际记录布局，不是内核速度的推测。

一千万点的计时案例与一百万像素的最终项目规模不同，二者的毫秒值不直接相除。它们共同说明：算法吞吐量和跨边界的数据量，要分别考虑。

10.5 看到颜色略有差别时

CPU 与 GPU 中心点的 radiance 分别为 (0.152368,0.172937,0.118156) 与 (0.152371,0.172865,0.118214)。这里展示的是单个采样点，并未声称全部结果逐位一致。

该程序含 normalize、sqrt、pow 和浮点表达式。两个后端采用的原生计算路径可以产生舍入差异。项目需要结合输出含义决定精度要求；金额累计、几何命中和图像高光，不能机械地使用同一个容许误差。

10.6 测量应服务于决定

想回答“换设备是否划算”，就测输入到所需输出的完整路径；想回答“驻留计算是否更快”，就使用已经准备好的资源；想回答“哪个阶段昂贵”，才进一步观察阶段时间。

配套日志保存一次运行的全部读数，便于核对图表。单次观测会受系统负载、调度与缓存影响；比较正式项目时，在相同输入和相同优化构建下重复单次测量，报告中位数及波动，保持准备、计算与传输的口径不变。

11 资产读取：让 Rust 并发服务于真实消费者

11.1 为什么 I/O 没有写成 B parallel

文件读取涉及路径、格式、操作系统和错误。它的工作对象不是一个规则数值域。BLEngine 使用 Rust 原生线程读取几何，将解码后的普通数据交给设备所有者，再由后者创建 GPU 资源。

这样，读取线程不需要拥有场景，也不需要复制设备状态。每个线程只生产一份带资产编号的 BLMesh。



图 11-1 文件完成顺序可以不同；资产编号保留身份，GPU 对象仍在其所属线程创建。

11.2 先看生产者的主干

src/geometry_io.rs 使用一个共享任务表和原子下标，令多个工作者各取一项。以下保留实际分工，略去线程命名和错误上下文格式：

Rust / 宿主代码

```
while let Some((key, path)) =
    sources.get(
    next.fetch_add(1, Ordering::Relaxed)
) {
    let result =
        BLMesh::read_cooked(path)
        .map(|mesh| (*key, mesh));
    if sender.send(result).is_err() {
        break;
    }
}
```

fetch_add 分配互不重复的任务编号。Relaxed 足以表达这里的编号领取；任务表在创建线程前已经准备好且保持不可变，结果通过通道交接。

发送失败意味着接收端已不存在，生产者没有后续消费者，应结束循环。这是实际生命周期的终点，不是失败后重新读取或伪造一个空模型。

11.3 容量为零的通道有什么用

实现使用 `mpsc::sync_channel(0)`。容量为零表示交接时发送者与接收者会合；工作者完成一份模型后，等待消费者接走，避免在消费者赶不上时继续积累许多已解码的大模型。

零容量不等于整个项目内存为零：每个工作者可以持有自己正在读取或等待交接的模型，消费者也拥有已经接走的数据。并发预算决定同时进行的实际读取数量。

11.4 接收结果时保留资产身份

下面是 src/scene.rs 消费几何读取结果的组织节选，省略材质分组装配：

Rust / 宿主代码

```
for source in load(requested)? {
    let (key, mesh) = source?;
    self.insert(
        device, program, key, &mesh,
        groups, needs_uv,
    )?;
}
```

结果按完成顺序到达。不能把“第一个返回的模型”当作“请求表第一个资产”；key 才是关联身份。真实实现还维护本次请求集合，保证交付对象与请求匹配；这是外部生产者交接的责任。

requested、groups、needs_uv 来自前面的请求与材质装配；节选省略了对应容器定义。

11.5 退出顺序也属于算法的一部分

若消费者提前结束，工作者可能正阻塞在 send。此时先 join 工作者，再释放接收端，可能令双方互相等待。

真实 Drop 顺序是先关闭接收端，再等待工作者：

Rust / 宿主代码

```
self.receiver.take();
if let Err(e) = self.finish() {
    eprintln!("{e}");
}
```

take 释放唯一接收端，阻塞的 send 随之结束；finish 再 join 工作者并报告线程异常。



图 11-2 先解除交接等待，再等待工作者结束；不把仍在读取的线程分离出去。

11.6 并发不需要成为常驻轮询

迭代入口用阻塞 recv 等待结果；非阻塞 try_next 的 Pending 交给已有事件循环处理，表示此刻未就绪。没有新请求时不创建读取工作，也无需常驻轮询。

12 一帧渲染：把资源交出去，再完整接回来

12.1 世界与输出拥有不同的寿命

一份世界可以产生多个输出：主窗口、预览图、离线图片。模型、材质和灯光属于共享世界；当前图像、叠加层和相机输入属于具体输出。

截至本书所固定的实现，BLEngine 用 BLSceneRenderer 持有世界资源，BLRenderOutput 持有某个输出的数据。新建输出不会复制一份世界，也不需要保存无消费者的默认图像。



图 12-1 两个输出可以消费同一世界；每个输出拥有自己的像素与视图输入。

12.2 B 把一帧写成两段

render/frame.b 的主体调用真实 trace，再调用 resolve。下面保留完整调用参数，省略已在接口声明的长参数表：

B / 语言代码

```
trace<N,W,H,S>(
    pixels,instances,overlays,surfaces,
    normal_matrices,cameras,lights,
    lighting,scene,overlay_scene,albedos,
    floor_present,ground_z,ground,
    overlay_start
);
resolve<W,H,S,DC,IC>(pixels,display);
```

trace 根据相机与场景进行光线查询和着色，写出线性 radiance；resolve 完成采样汇合、显示映射和像素打包。DC 是显示像素数量，IC 是内部采样数量，S 是每轴内部采样倍数。

Rust 不需要在两段之间读取 pixels，也不再实现另一套显示公式。依赖已经包含在 B program 内，设备执行据此衔接。

12.3 看懂一帧的九个数据绑定

这个已提交版本中，普通 dataset 绑定如下。加速结构与纹理另有资源槽位，不能混入这个 Vec 当作 Dataset。

绑定	数据	提交前的所有者
0	pixels	当前输出
1	display	当前输出
2	instances	世界输入
3	overlays	当前输出
4	surfaces	世界输入
5	normal_matrices	世界输入
6	cameras	当前输出
7	lights	世界输入
8	lighting	世界输入

这张表是具体 program 的接口，不应推广成所有渲染器都必须使用的固定编号。项目改变程序参数时，应同时调整其真实宿主装配。

12.4 mem::take 不是复制

摘自 src/output.rs，保留一次移交的实际过程：

Rust / 宿主代码

```
let mut datasets =
    std::mem::take(&mut self.data);
let cameras = datasets.remove(3);
let overlays = datasets.remove(2);
datasets.append(&mut inputs.data);
datasets.insert(3, overlays);
datasets.insert(6, cameras);
```

self.data 最初依次持有 pixels、display、overlays、cameras。take 把整个 Vec 移出，原位置留下空 Vec；append 把世界数据移入；insert 再按 program 接口放回相机与叠加层。

这里只移动拥有 GPU 资源的 Rust 对象，没有复制像素或模型内容。提交期间，实际绑定由提交对象拥有。

12.5 提交时连同加速结构一起交接

Rust / 宿主代码

```
let done = self.program.submit(
    Bindings {
        datasets,
        accelerations: std::mem::take(
        &mut inputs.accelerations
        ),
        textures: inputs.textures.clone(),
    },
    &values,
)?.wait()?;
```

纹理 clone 共享实际纹理存储；加速结构随本次提交一起移动。光线查询需要 TLAS，TLAS 又依赖 BLAS；实际 GPU 消费关系必须保留，不能用 CPU 一帧结束作为统一释放点。

12.6 完成后分回原来的所有者

Rust / 宿主代码

```
let (bindings, accelerations, timings) =
    done.into_parts();
let mut data = bindings.into_datasets();
let cameras = data.remove(6);
let overlays = data.remove(3);
inputs.data = data.split_off(2);
data.extend([overlays, cameras]);
self.data = data;
inputs.accelerations = accelerations;
```

这一段与提交前的表正好对应。世界拿回 2、4、5、7、8，输出拿回 0、1、3、6，下次即可继续使用原来的存储。



图 12-2 所有权移动发生在主机，像素计算发生在设备；完成后数据回到各自的长期消费者。

如果提交或等待返回 Err，本次绘制失败，应沿错误路径交给调用者；不能继续假定此前移出的向量仍完整存在。这里没有额外的恢复状态机。

12.7 读回和显示位于最后一层

BLRenderOutput::save 只下载 display，写出 PPM。原生呈现路径则可以直接消费设备显示数据。两种消费者的需求不同，不能为了“方便调试”让每个实际显示帧都下载完整中间结果。

frame 还需要真实几何、相机、纹理和光线查询资源。本书附录提供该入口的编译检查命令；它确认源程序能够生成执行阶段，不冒充完整场景已经原生渲染。下一章的独立项目则提供完整可运行的图像交付。

13 完整项目：生成一幅材质图

13.1 把学过的关系接起来

最终项目输出 1000×1000 像素，即一百万像素的材质图。B 负责生成采样点、计算真实 GGX、整理 radiance，并调用引擎真实 resolve；Rust 持有设备、数据和纹理，只在最终图像完成后读回并保存。

它没有光线查询或完整场景，也没有伪装成完整引擎渲染。这个项目专注一条从数学计算到实际文件的完整混合编程链路。



图 13-1 B 的中间计算留在设备，Rust 在最终文件边界处理字节与 I/O。

13.2 项目目录就是一张职责图

执行关系 / 示意

```
sample/hybrid/
  Cargo.toml
  src/
    main.rs        命令与布局读写
    particles.rs   粒子与提交组合
    scene.rs       场景输入与结果解释
    sheet.rs       纹理、设备、文件
  Programs/
    book/          教学组合
    simulation/    引擎原文
    scene/         引擎原文
    material/      引擎原文
    render/        引擎原文
    math/          引擎原文
    lighting/      引擎原文
```

Cargo.toml 只声明 blanguage 与 blcompute 的本仓库路径依赖。Programs 不依赖 B:\BLEngine，也没有把引擎 Rust 工程反向纳入语言编译器。

13.3 B 组合程序保留最重要的四步

book/sheet.b 的 image 正文：

B / 语言代码

```
materials<1000,1000>(points,albedo);
pack(points,pixels);
resolve<1000,1000,1,1000000,1000000>(
    pixels,display
);
```

第一行内部包含 prepare 与真实 shade。pack 只整理输出字段，让两种真实引擎记录类型衔接：

B / 语言代码

```
program pack(
    read dataset<BLShadingPoint,1000000>
        points,
    write dataset<BLPixel,1000000> pixels
) on compute {
    parallel (pixel in pixels) {
        pixel.radiance =
        points[index()].radiance;
    }
}
```

输入与输出具有相同静态长度，index() 在这个读取范围中成立，不需要 unsafe_read。单独的 read 参数也清楚表达 pack 不修改 points。

prepare、shade、pack、resolve 是源程序层次的四步，不意味着最终必然生成四个 GPU 内核。编译器可根据依赖与可融合性合并阶段；查看编译输出才能知道当前实际生成的执行阶段数。

13.4 显示转换为什么也是 B 算法

render/resolve.b 对每个显示像素累积 S×S 个内部 radiance，再映射到显示值。此项目 S=1，因此每个显示像素读取一个内部像素。

显示映射的核心为：

B / 语言代码

```
float3 mapped = pow(
    max(radiance/(radiance+1.0),
        float3(0.0)),
    0.454545454545
);
```

随后每个通道乘 255、舍入并限制到字节范围，最后打包：

B / 语言代码

```
pixel.bgra = (red<<16) | (green<<8)
    | blue | 0xff000000;
```

这部分 max 和 clamp 定义显示映射与输出格式，属于算法语义。删除它们可能改变输出或整数转换结果；主干化不等于任意删除数学操作。

13.5 Rust 创建实际设备资源

以下对应 sheet.rs 的 image 函数，辅助 program 函数在第二章已经介绍：

Rust / 宿主代码

```
let program = program(
    "book/sheet.b", "image"
)?;
let device =
    Device::for_program(&program)?;
let data = (0..3)
    .map(|i| {
        device.zeroed(
        &program,i,1_000_000
        )
    })
    .collect::<Result<Vec<_>>>()?;
let albedo = device.upload_texture(
    &texture()?
)?;
let program = device.compile(program)?;
```

三个绑定依次为 points、pixels、display。texture 函数创建 2×2 的浮点颜色纹理，具体四个 RGBA 值见配套 sheet.rs。它是真实可采样资源，示例无需额外下载图片。

device.zeroed 和上传可以留下尚未完成的设备操作，后续执行框架沿真实依赖衔接它们。应用不需要为每项创建后都插入一次主机 wait。

13.6 一次提交，到最终结果才等待

Rust / 宿主代码

```
let done = program.submit(
    vulkan::Bindings {
        datasets: data,
        accelerations: vec![],
        textures: vec![albedo],
    },
    &[],
)?.wait()?;

let display = device.download(
    &done.datasets()[2]
)?;
```

这里唯一读回的是 display。没有把一百万条 112 字节的点记录或 16 字节的内部 radiance 记录复制回 Rust，交接的数据正好对应最后的文件消费者。

13.7 文件边界保留明确的错误传播

PPM 是一个简单、无需额外图像库的 RGB 格式。Rust 负责写头、拆 BGRA 字节并保存：

Rust / 宿主代码

```
file.write_all(b"P6\n1000 1000\n255\n")?;
for &bgra in display.words() {
    file.write_all(&[
        (bgra >> 16) as u8,
        (bgra >> 8) as u8,
        bgra as u8,
    ])?;
}
file.flush()?;
```

实际源码使用 BufWriter 包装 File。flush 的错误同样沿 Result 返回，只有文件写入完成后才输出图像路径。把格式字节写入文件是 Rust 的职责，色调映射已经由 B 完成。

13.8 编译、运行和打开图像

PowerShell / 命令行

```
$env:CARGO_TARGET_DIR = `
    Join-Path $PWD 'target'
$manifest = 'sample/hybrid/Cargo.toml'
cargo build --release `
    --manifest-path $manifest

& $book image
```



图 13-2 完整 B 图像程序的设备时间为 2.176 ms；含提交和等待的执行为 7.172 ms，显示下载为 11.708 ms。

输出位于 Saved/HybridBook/materials.ppm。使用支持 PPM 的图像查看器打开即可；书中的预览由该文件转换为 PNG，仅用于书页显示。

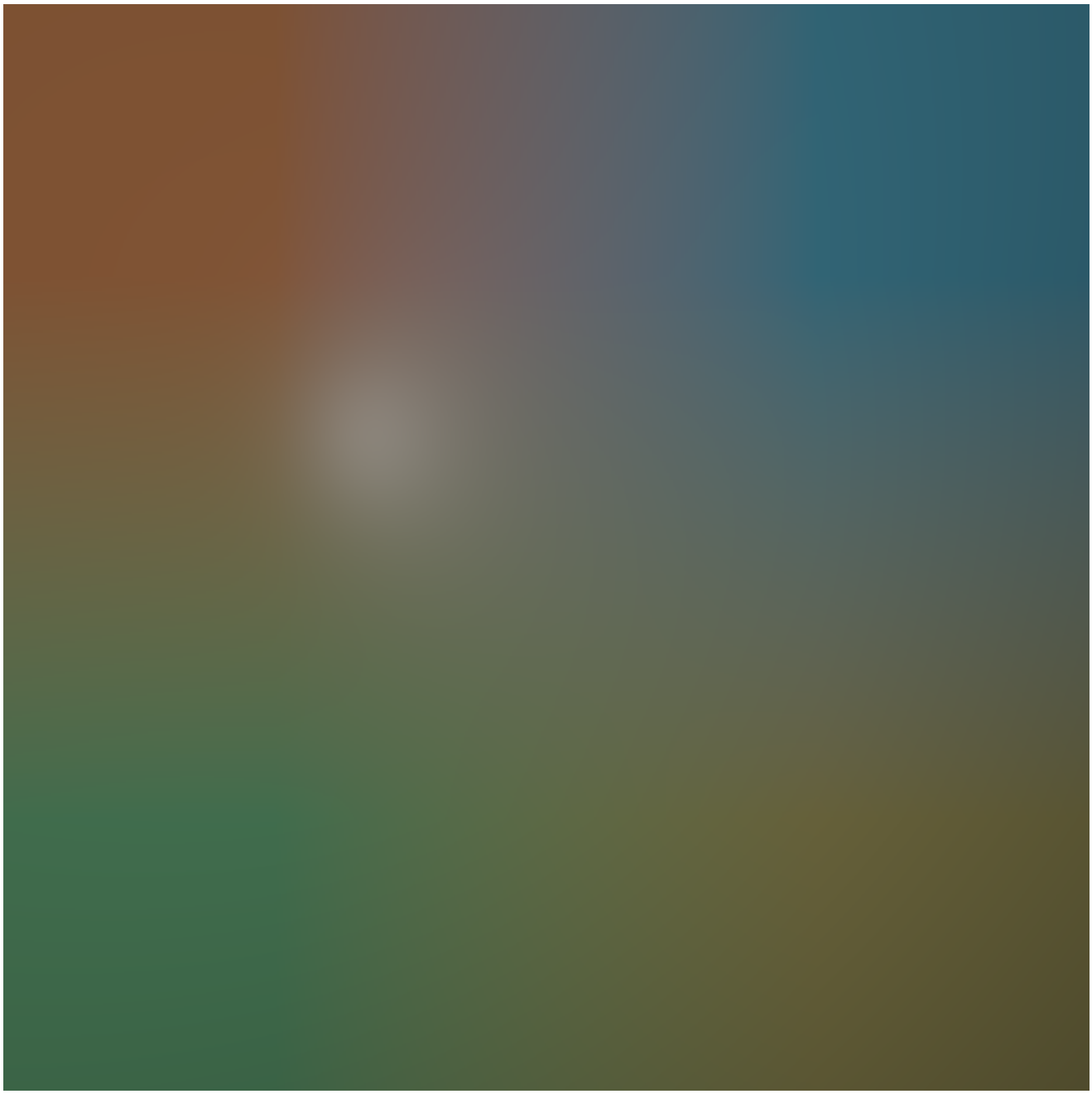


图 13-3 采样位置改变法线、纹理坐标和粗糙度，形成连续的表面响应。这是独立材质采样图。

观察从左到右的变化，再回到 prepare 中的 roughness 公式。像素亮度同时受法线、纹理与高光影响，不能只凭某处更亮就推断其粗糙度更小。

13.9 把它接到自己的项目里

应用长期持有 Device、GpuProgram、纹理和可复用存储；输入发生变化时更新相应数据，再按实际消费链提交。如果最终消费者也是 GPU，就继续传递设备结果；只有需要 CPU 的业务结果或文件输出时才下载。

14 走向复杂项目：保持边界，增加能力

14.1 让模块围绕实际概念生长

BLEngine 的 B 模块按 math、simulation、scene、material、lighting、render 组织；Rust 按场景、资产、设备与输出的实际所有者组织。两边的目录无需逐文件镜像，但同一概念的归属应当清楚。

添加一个新算法时，先找它的输入、输出与消费者。比如新的碰撞计算需要几何和状态，可以由 B 提供数值阶段；Rust 场景继续维护对象身份与生命周期。不要为了调用方便，让算法持有整个编辑器或复制整个世界。

14.2 把编译期、加载期和执行期分开

W、H、纹理数组长度等静态形状决定具体 program；加载后的纹理和模型具有资源寿命；dt、强度和相机状态则可能随执行变化。

静态参数改变时可能需要新的具体程序；只改变 dt 时无需重新解析所有模块；没有改变的模型也无需重复读取文件。把对象放在与其实际寿命一致的所有者中，比为每一步增加缓存层更直接。

src/hierarchy.rs 的 BLScenePrograms 保存实际使用的 CpuProgram，按需要准备相应入口。它拥有可复用的程序，不拥有第二份场景事实源。

14.3 原生 Rust API 与 DLL 如何选择

集成方式	适合的消费者	当前能力
blanguage + blcompute	源码级 Rust 项目	数据所有权、view、纹理、设备驻留、提交组合
blc build-dll	C/C++ 项目或独立二进制接口	DLL、导入库、C/C++ 头与 Rust 绑定，纯值数据及 uniform

Rust 无需经过 C ABI 才能使用 B。原生接口保留了本书的资源模型；需要向现有 C++ 产品发布一个独立算法 DLL 时，使用同一算法的 DLL 路径即可。第一本书与 Docs/NativeInterop.md 给出完整构建和调用实例。

当前 DLL 的同步调用不表达本书完整的设备资源与异步组合，不能用原生 API 的能力描述替代其实际 ABI 范围。B 源码目前也没有任意导入 C++ 或 Rust 外部函数的语法；常见组织方式是 Rust/C++ 宿主调用 B，然后继续调用各自生态。

14.4 复杂度应留在真正需要的地方

输入资产有文件格式和拓扑边界，场景有身份与无环父关系，GPU 有设备归属和真实完成点。它们保护的是不同所有者之间的交接。

一个普通数学函数不应为“以防万一”再建立一套状态、重复校验或后台巡检。数据既已由明确所有者产生，消费者就沿正式接口使用它；出现问题时追到真实生产者，直接修正关系。

这里保留的错误处理都有具体含义：文件无法读取，程序无法编译，资源无法创建，消费者提前结束，设备执行失败。它们不能用一个空结果或默认图像悄悄替换。

14.5 三个可以继续完成的改造

第一个改造：把材质图的 roughness 区间变为运行时参数。保持 W、H 的静态形状，修改 B 接口与实现中的 uniform，再由 Rust 按顺序传入 Value::F32。观察只改参数时能否继续复用程序与纹理。

第二个改造：把教学四颜色纹理换成自己的 .bltex 资产。Rust 负责读取 Texture2D，再上传到原有程序；B 仍只接收 read texture2d。材质公式无需知道文件来自本地磁盘还是其他正式资产来源。

第三个改造：将一个材质结果分别输出成图片和下游 GPU 输入。先画出两个消费者，决定是否独立输出、是否共享只读来源，再选择提交组合。不要先下载到 CPU，只因为两个消费者位于不同 Rust 函数中。

14.6 回看同一条主线

从粒子的一个时间步，到两组结果的汇合，从节点世界变换到相机与灯光，从共享纹理到最终像素，算法始终沿数据依赖前进。Rust 的所有权则跟着实际存储一起交接。

当两条关系能够互相对应时，复杂项目也能保持清晰：每项计算有明确输入，每份资源有真实消费者，每次等待都有原因。这样的结构，才让并行能力在工程中持续发挥作用。

附录 A 完整命令与运行记录

A.1 一次准备，逐章运行

以下命令在 x64 Developer PowerShell、Blink 根目录执行：

PowerShell / 命令行

```
cargo build --workspace --release
$env:CARGO_TARGET_DIR = `
    Join-Path $PWD 'target'
$manifest = 'sample/hybrid/Cargo.toml'
cargo build --release `
    --manifest-path $manifest

$book = `
    'target/release/blink-rust-book.exe'
& $book particles cpu 3
& $book particles gpu 3
& $book steps
& $book flow
& $book scene cpu
& $book scene gpu
& $book sheet cpu
& $book sheet gpu
& $book image
```

所有命令会直接传播失败；GPU 命令不会在设备不可用时改成 CPU 执行。sheet 使用约 1.12 GB 的点记录，加上执行与传输所需资源；image 使用一百万像素的独立数据，二者规模不同。

A.2 单独查看 B 编译结果

PowerShell / 命令行

```
$blc = '.\target\release\blc.exe'
$root = 'sample/hybrid/Programs'
& $blc check `
    "$root/simulation/statistics.b" `
    $root step_and_total

& $blc check "$root/book/sheet.b" `
    $root 'materials<10000,1000>'

& $blc check "$root/book/sheet.b" `
    $root image
```

一千万采样点的 materials 在本书版本中具有两个源阶段，优化后为一个执行 Map 阶段；这也解释了为什么不能仅数源程序调用就判断内核数量。

A.3 真实引擎 frame 的编译检查

PowerShell / 命令行

```
& $blc check "$root/render/frame.b" `
    $root `
  'frame<1,1000,1000,1,1000000,1000000>'
```

本书原生检查得到两个执行阶段：trace 写 pixels，resolve 读 pixels 写 display。dataset 步长依次为 16、4、80、48、40、48、80、112、48 字节；此外还有两个 acceleration 和一个纹理数组。

这个命令不创建场景、不构造光线加速结构，也不输出 frame 图像。它用于阅读真实编译接口；完整可运行图像项目使用 A.1 的 image。

A.4 保存结果与阅读数值

配套 sample/hybrid/运行记录.txt 保存书中原生运行的终端文本，图中的数值由这些记录排印。输出图像路径为 Saved/HybridBook/materials.ppm。重新运行时耗时会变化，程序不会为了吻合书中数字而固定结果。

particle、steps 和 flow 的显示小数用于理解算法；CPU/GPU 材质输出存在浮点差异，书中只展示实际观察到的中心点。需要数值精度约束的项目应围绕自己的正式输出要求设计计算。

附录 B 源码阅读路线与摘录范围

B.1 B 算法与 Rust 宿主相互对照

学习问题	B 源文件	BLEngine Rust 源文件
粒子推进与能量	simulation/bodies、statistics	src/main.rs 的模拟入口
世界变换	scene/hierarchy	src/hierarchy.rs
相机与灯光	scene/components	src/components.rs
共享几何与 view	render/trace.h	src/scene.rs
纹理与 GGX	material/textured、surface	src/material.rs、render.rs
文件生产与交接	数据进入计算之前	src/geometry_io.rs
一帧与最终显示	render/frame、resolve	src/render.rs、output.rs

路径中的 B 模块同时对应 .h 与 .b。sample/hybrid/Programs 保存本书使用的引擎模块及其导入闭包；BLEngine摘录来源.md 列明每个文件的已提交版本和哈希，BLEngine-LICENSE.txt 保留其 MIT 授权。

B.2 为什么保留导入闭包

resolve 使用 render.trace 定义的 BLPixel，因此教学项目同时携带 trace 接口及其依赖。文件存在不表示每个入口都要执行完整光线查询；具体 program 的资源需求和实际阶段以所选入口的编译结果为准。

教学 book 模块只生成输入、连接记录与演示汇合。它没有另写一套 GGX 或显示映射来替代引擎实现。

B.3 阅读节选时先找五个位置

先读接口的参数与权限，再找数据来源；接着看 program 的阶段组合，沿 Rust 的实际绑定追到执行；最后看完成后的消费者和释放位置。

遇到节选省略的容器或打包过程，可以在配套宿主中先掌握同类完整写法，再回到引擎路径阅读其业务组织。不要把省略符当作可以直接编译的完整程序。

B.4 关于后续引擎演进

BLEngine 仍在发展。本书固定一个已提交版本讲述已存在的计算和资源关系，后续 Ultra 等渲染实现可能改变具体阶段、参数或绑定编号。学习时应保留本书的职责与寿命方法，并以所使用源码的实际接口为准。

Copyright (c) 2026 Blink contributors。引擎摘录 Copyright (c) 2026 BLEngine contributors，MIT License。正文与配套教学源码随 Blink 开源仓库发布。